

集合

- **22-1:陣列與集合的比較**
 - 何謂集合.....22-1
 - 比較陣列與集合.....22-2
- **22-2:傳統集合**
 - 傳統集合.....22-3
 - 建立ArrayList.....22-4
 - 集合常用的屬性與方法.....22-5
- **22-3:Generic泛用集合**
 - 泛用(Generic)集合.....22-8
 - Generic泛用的優點.....22-9
 - 使用List<T>.....22-10
 - List<T>與class的應用.....22-11
 - List<T>使用foreach迴圈.....22-13
 - 字典集合Dictionary<TKey,Tvalue>.....22-14
 - 使用Dictionary<TKey,Tvalue>.....22-15
- **22-4:傳統集合與泛用集合**
 - 傳統集合與泛用集合.....22-16

Module 22

何謂集合

- 陣列在記憶體中占用的是一個連續的位置，且在建立時必須先定義長度，建立後無法無法直接新增、刪除或插入元素，因為會影響其他物件的記憶體位置，若要改變長度，必須重新建立新的陣列物件並複製資料。
- 集合正好解決了這類問題，集合建立時不須先定義長度，在記憶體中非連續性，可隨需求動態新增或移除，開發者可藉由集合更方便地管理多筆資料，減少手動調整陣列大小的麻煩。

比較陣列與集合

	陣列	集合
長度	固定長度	可變長度
資料型態	只能存放相同資料型態的元素	非Generic集合可以存放不同的資料型態
唯讀	資料無法製成唯讀	非Generic集合的元素可製成唯讀
彈性	較不佳	較佳
執行效能	較佳	較不佳

傳統集合

- 傳統集合為非泛用集合。
- 非泛用集合內的元素型別都是物件。
- 內容值可為任意型態(object)。
- 取用資料時，會觸發Boxing(裝箱)或執行Unboxing(拆箱)機制。
 - Boxing：將資料轉換成object型別。
 - Unboxing：從object型別擷取資料型別。

建立ArrayList

- 宣告及建立ArrayList：

```
ArrayList myList = new ArrayList();
```

- 為ArrayList加入元素 - .Add()方法：

```
myList.Add("I'm still same.");  
myList.Add("都好像沒有bang");  
myList.Add("Nothing Changed.");
```

集合常用的屬性與方法-1

- **Count**屬性：
 - 集合的元素個數。

```
Console.WriteLine(myList.Count); // 3
```
- **ListObject.Insert(int index, object val)**方法：
 - 在指定位置(index)插入元素(val)。
- **ListObject.Remove(object val)**方法：
 - 移除集合中第一個指定的元素(val)。
- **ListObject.RemoveAt(int index)**方法：
 - 移除集合中指定的索引位置(index)。

集合常用的屬性與方法-2

- **ListObject.Clear()**方法：
 - 清除所有集合中的元素。
- **ListObject.Sort()**方法：
 - 將集合中元素排序。
- **ListObject.Reverse()**方法：
 - 反轉集合中元素順序。

集合常用的屬性與方法-3

- **ListObject.AddRange(ICollection c)**方法：

- 一次加入多個元素到集合。

```
myList.AddRange(new int[] { 1, 2, 3, 4 });
```

- **ListObject.RemoveRange(int index, int count)**方法：

- 移除指定索引(index)後的多個(count)元素。

```
myList.RemoveRange(0, 3);
```


泛用(**Generic**)集合

- 泛型 (**Generic**) 被廣泛應用於集合 (**Collection**) 中。
- 相較於傳統的非泛型集合 (如 **ArrayList**、**Hashtable**)，泛用集合在處理資料時具有更高的效能與安全性。
- 不需裝箱 / 拆箱 (**Boxing / Unboxing**)。
 - 資料型態一致，避免轉換成本。
- 可在編譯階段發現型別錯誤，降低執行期例外。
- 提升應用程式效能與可靠性。

Generic泛用的優點

- 重複使用性：
 - 一個泛用類別可建立多種不同型態的物件。

```
List<int> myintList = new List<int>();  
List<string> mystringList = new List<string>();  
List<MyClass> myclassList = new List<MyClass>();
```

- 型別安全性：
 - 自動檢查型別，避免執行時發生錯誤。
- 高效率：
 - 元素資料不須轉型Boxing與Unboxing。

使用List<T>

- 是常用的泛用型別。
- 宣告、建立及使用的方法與ArrayList大致相同。

```
List<int> myintList = new List<int>();  
myList.Add(1);  
myList.Add(2);  
myList.Add(3);  
for(int i = 0; i < 10; i++)  
{  
    myList.Add(i);  
}
```

List<T>與class的應用-1

- 泛用集合List可為任意型別，可以建立自訂物件的集合。

```
public class clsProduct
{
    public string Name;
    public string Developer;
    public int Price;
}
```

```
clsProduct ns = new clsProduct()
{
    Name = "Nintendo Switch", Developer = "Nintendo", Price = 9780
};
clsProduct ps4 = new clsProduct()
{
    Name = "Play Station 4 PRO", Developer = "SONY", Price = 12980
};
List<clsProduct> LsProducts = new List<clsProduct>();
LsProducts.Add(ns);
LsProducts.Add(ps4);
```

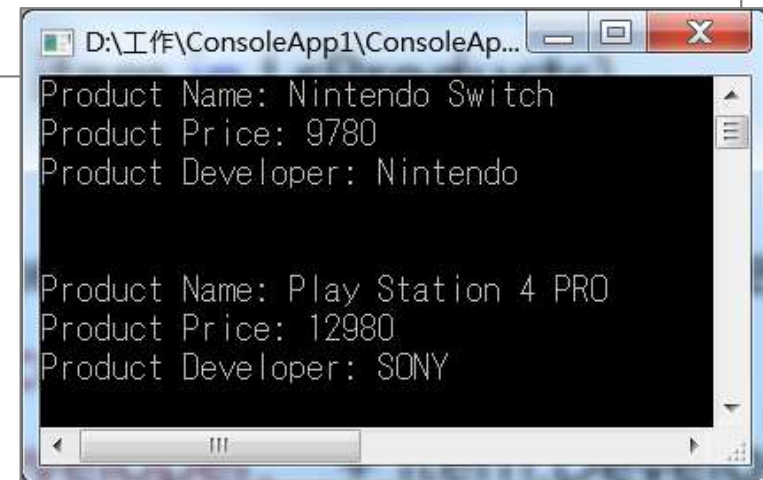
List<T>與class的應用-2

- List初始化：

```
clsProduct ns = new clsProduct()
{
    Name = "Nintendo Switch", Developer = "Nintendo", Price = 9780
};
clsProduct ps4 = new clsProduct()
{
    Name = "Play Station 4 PRO", Developer = "SONY", Price = 12980
};
List<clsProduct> LsProducts = new List<clsProduct>()
{
    ns,
    ps4
};
```

List<T>使用foreach迴圈

```
foreach(clsProduct item in LsProducts)
{
    Console.WriteLine("Product Name: " + item.Name +
        "\nProduct Price: " + item.Price +
        "\nProduct Developer: " + item.Developer + "\n\n");
}
```



The screenshot shows a console window titled "D:\工作\ConsoleApp1\ConsoleAp...". The output displays two product entries, each with its name, price, and developer on separate lines, followed by a blank line. The first entry is for "Nintendo Switch" with a price of 9780 and developer "Nintendo". The second entry is for "Play Station 4 PRO" with a price of 12980 and developer "SONY".

```
Product Name: Nintendo Switch
Product Price: 9780
Product Developer: Nintendo

Product Name: Play Station 4 PRO
Product Price: 12980
Product Developer: SONY
```

字典集合 **Dictionary<TKey,Tvalue>**

- 字典集合有兩個元素，分別為**Key**與**Value**。
 - Key表示為索引值。
 - Value為對應Key的值。
- Dictionary<TKey,Tvalue>**的**Key**與**Value**可為任意型別。

```
Dictionary<int, string> dic2 = new Dictionary<int, string>();
```

```
Dictionary<string, int> dic1 = new Dictionary<string, int>();
```

- Dictionary<TKey,Tvalue>**使用的方法與**List<T>**大致相同。

使用 Dictionary<TKey,Tvalue>

```
Dictionary<string, int> dic1 = new Dictionary<string, int>();  
dic1.Add("Switct", 9780);  
dic1.Add("PS4 PRO", 12980);  
Console.WriteLine(dic1["PS4 PRO"]); //12980
```

```
Dictionary<string, int> dic1 = new Dictionary<string, int>()  
{  
    { "Switct", 9780 },           //初始化  
    { "PS4 PRO", 12980 }  
};  
Console.WriteLine(dic1["PS4 PRO"]); //12980
```


傳統集合與泛用集合-1

傳統集合

```
ArrayList myList = new ArrayList();  
myList.Add(1);           // Boxing  
myList.Add("2");         // 應要產生錯誤警示  
int i = (int)myList[0];  // 須轉型Unboxing
```

泛用集合

```
List<int> myList = new List<int>();  
myList.Add(1);           // 未Boxing  
myList.Add("2");         // 產生錯誤警示  
int i = myList[0];       // 不須轉型
```

傳統集合與泛用集合-2

傳統集合	泛用集合
ArrayList	List<T>
Hashtable	Dictionary<TKey, TValue>
SortedList	SortedList<TKey, TValue>
Queue	Queue<T>
Stack	Stack<T>
IEnumerable	IEnumerable<T>
ICollection	N/A(可用IEnumerable<T>模擬)
N/A	ICollection<T>
IList	IList<T>
CollectionBase	Collection<T>
ReadOnlyCollectionBase	ReadOnlyCollection<T>
DictionaryBase	N/A(可用IEnumerable<T>模擬)
N/A	SortedDictionary<TKey, TValue>
N/A	KeyedCollection<TKey, TValue>
N/A	LinkedList<TKey, TValue>